

Clase 1 — Monolito y Microservicios

- Clase 1 — Monolito → Microservicios + Estrategias de migración
 - Introducción
 - Términos necesarios para este apunte
 - 1. El problema de escalar un monolito
 - 2. El monolito
 - 3. Microservicios
 - 4. Trade-offs: cuándo migrar (y cuándo no)
 - 5. Estrategias de migración
 - 6. EventOS: aplicación práctica
 - 7. Trade-offs y decisiones arquitectónicas
 - 8. Puntos clave
 - 9. Referencias y profundización

Clase 1 — Monolito → Microservicios + Estrategias de migración

ARQ 2026 · UCC

Introducción

Todo sistema de software exitoso enfrenta en algún punto la misma pregunta: ¿cómo seguir creciendo sin que la arquitectura se convierta en un obstáculo?

Construir un sistema que funciona es un problema resuelto. El desafío real está en hacer que ese sistema evolucione: que pueda escalar cuando el tráfico crece, que un equipo de 20 personas pueda trabajar sobre él sin pisarse, que un bug en un módulo no derribe todo lo demás.

Este apunte estudia esa transición: cuándo y por qué un sistema monolítico empieza a mostrar sus límites, qué ofrece la arquitectura de microservicios como alternativa, y cómo se ejecuta esa migración de forma incremental sin detener el sistema.

A lo largo del apunte se usa **EventOS** — una plataforma de venta de tickets online — como caso de referencia para ilustrar los conceptos en un contexto concreto.

Al finalizar la clase deberías poder:

- Identificar señales que justifican —o desaconsejan— migrar un monolito.

- Comparar los trade-offs entre un monolito modular y una arquitectura de microservicios.
- Explicar cómo se combinan estrategias de migración incremental para extraer una funcionalidad con riesgo acotado.

Términos necesarios para este apunte

- **ACID**: propiedades de una transacción de base de datos: atomicidad, consistencia, aislamiento y durabilidad.
 - **CI/CD**: prácticas y pipelines de integración continua y entrega o despliegue continuo.
 - **OOM (Out Of Memory)**: error producido cuando un proceso agota la memoria disponible.
 - **Runtime**: entorno que ejecuta una aplicación, como la versión de Go, Java o Node.js.
 - **Rollback**: vuelta a una versión o comportamiento anterior ante un problema.
 - **Cutover**: momento en que el tráfico o la operación pasa del sistema anterior al nuevo.
 - **Distributed tracing**: trazabilidad de una request a través de múltiples servicios mediante identificadores de correlación.
-

1. El problema de escalar un monolito

La mayoría de los sistemas de software empiezan como monolitos. Es la decisión correcta: un monolito es simple de construir, fácil de debuggear y barato de operar. La industria pasó décadas construyendo sistemas monolíticos exitosos.

El problema no es el monolito en sí — es lo que pasa cuando ese sistema crece. A medida que aumentan el tráfico, el equipo y la complejidad del negocio, ciertas características de la arquitectura monolítica — que antes eran invisibles — se convierten en fricciones concretas: deployments que tardan y afectan todo el sistema, módulos que se acoplan sin que nadie lo planifique, equipos que no pueden trabajar en paralelo sin pisarse.

Entender cuándo y por qué ocurre esa transición es el punto de partida para tomar decisiones arquitectónicas bien fundamentadas. Las estrategias de migración que se desarrollan más adelante son la respuesta a un problema de ingeniería real: cómo cambiar el motor de un avión en vuelo.

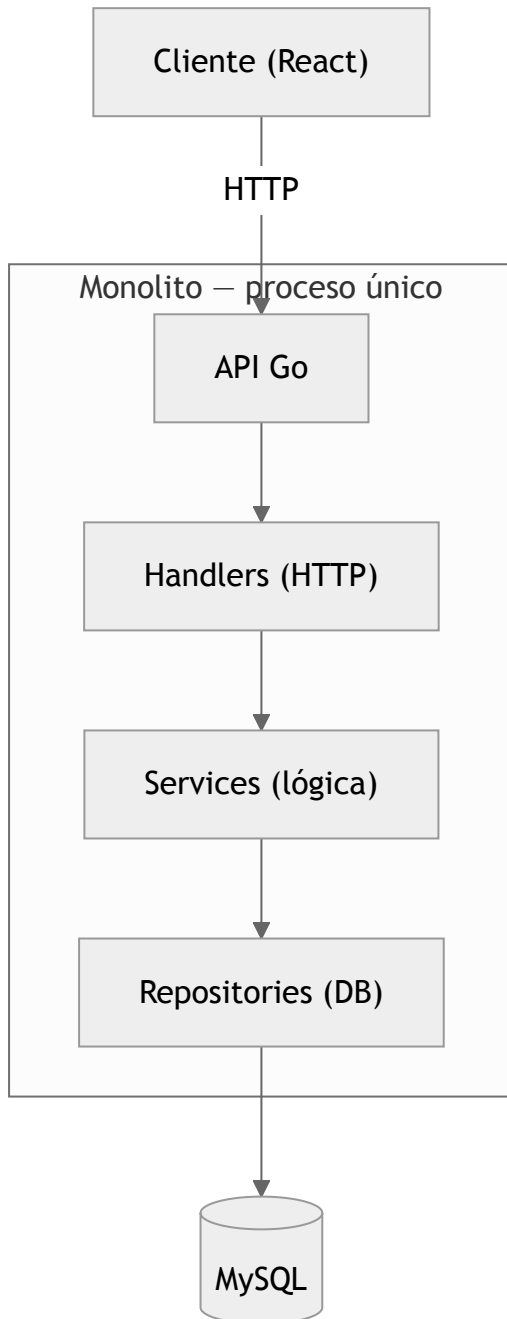
2. El monolito

2.1 Definición y características

Un **monolito** es una aplicación desplegada y versionada como una única unidad. En cada instancia de la aplicación, sus módulos — autenticación, lógica de negocio y acceso a datos — corren dentro del mismo

proceso y se despliegan juntos.

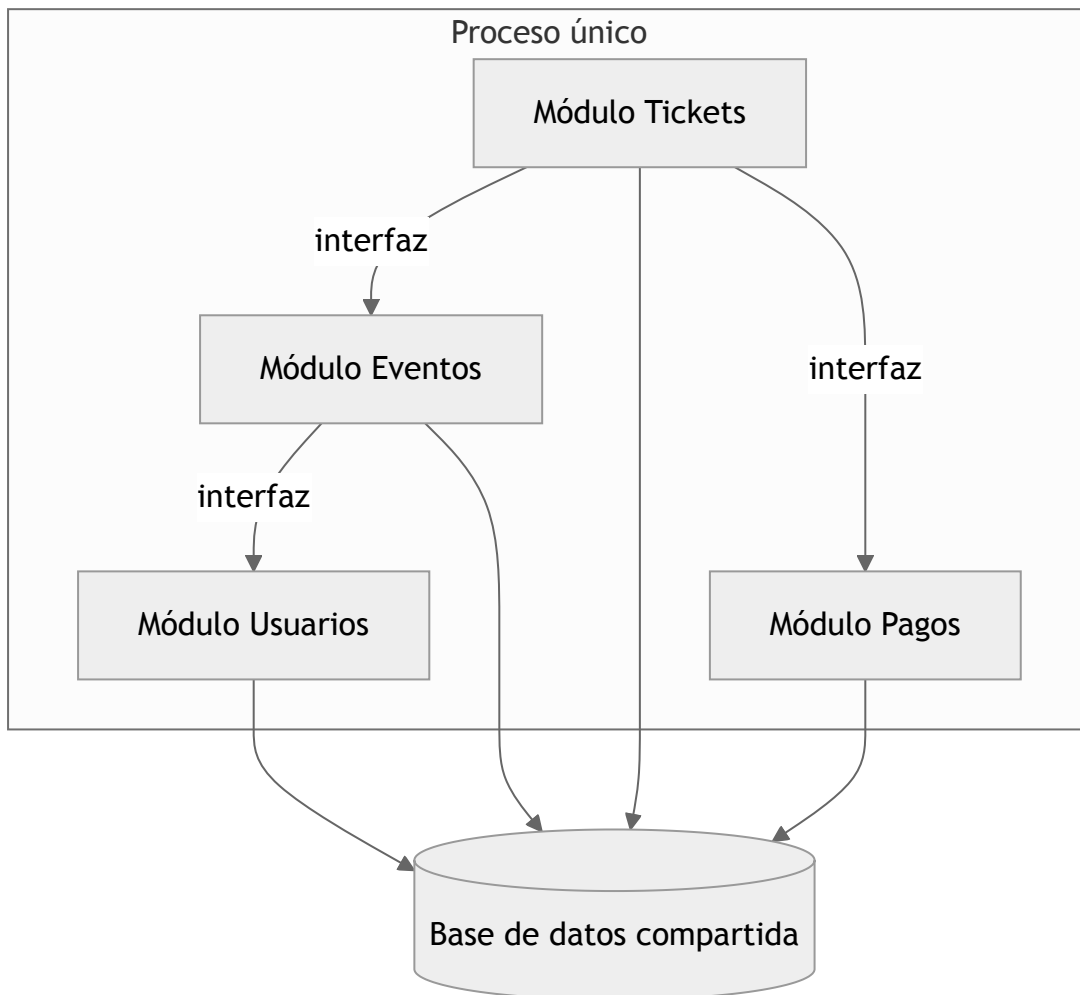
Un ejemplo típico es un servidor web que expone una API REST con una arquitectura por capas: los handlers reciben las requests HTTP, los services contienen la lógica de negocio, y los repositories acceden a la base de datos. Todo corre en un único proceso.



Lo importante de esta definición es lo que **no** implica: un monolito no tiene por qué estar mal organizado ni ser un desastre de código. Un monolito puede tener módulos bien separados, interfaces limpias entre capas y tests exhaustivos. El hecho de que todo corra en el mismo proceso es una característica de despliegue, no una condena de calidad.

2.2 El monolito modular

El **monolito modular** es una variante donde los límites entre módulos están explícitamente definidos dentro del mismo proceso. Cada módulo tiene su propia interfaz interna y los demás módulos solo pueden acceder a él a través de esa interfaz — no directamente a sus internals.



El monolito modular es una arquitectura legítima y frecuentemente subestimada. Para muchos sistemas, es la mejor opción: captura gran parte de los beneficios de la separación de responsabilidades sin la complejidad operacional de los sistemas distribuidos. Si el problema del monolito es el acoplamiento interno, la solución no necesariamente es distribuirlo — puede ser modularizarlo mejor.

La diferencia fundamental con los microservicios está en el límite de proceso y memoria: en un monolito modular, la separación entre módulos es *lógica* — ocurre dentro del mismo proceso de ejecución con memoria compartida, y el compilador o el runtime no impiden que un módulo acceda a los internals de otro si el desarrollador lo permite. En los microservicios, la separación es *física*: cada servicio corre en su propio proceso, con su propio espacio de memoria, y la única forma de comunicación es a través de la red. Esa frontera de red es lo que hace que las garantías de aislamiento sean reales y no dependan de la disciplina del equipo.

2.3 Ventajas del monolito

- **Simplicidad operacional:** un solo servicio para desplegar, monitorear y mantener. Un solo proceso para reiniciar cuando algo falla.
- **Desarrollo inicial rápido:** agregar una funcionalidad no requiere coordinar cambios en múltiples servicios ni definir contratos de API entre ellos.
- **Debugging directo:** el stack trace de un error es lineal. Se puede usar un debugger local, inspeccionar el estado completo de la aplicación, reproducir el problema fácilmente.
- **Sin latencia de red interna:** la comunicación entre módulos es una llamada de función — nanosegundos, no milisegundos.
- **Transacciones simples:** una operación que toca múltiples módulos puede ejecutarse en una única transacción de base de datos, con todas las garantías ACID.
- **Consistencia:** no hay riesgo de que dos módulos tengan versiones incompatibles del sistema en producción al mismo tiempo.

2.4 Límites a escala

A medida que el sistema crece — en funcionalidad, en tráfico, en tamaño del equipo — el monolito empieza a mostrar fricciones que no eran visibles al principio.

Acoplamiento fuerte

En la práctica, los módulos de un monolito tienden a acoplarse con el tiempo. Un módulo llama directamente a los internals de otro, comparte estructuras de datos, o depende de comportamientos no documentados. Cuando eso sucede, un cambio en un módulo puede romper otro aparentemente no relacionado. El sistema se vuelve frágil: cada modificación requiere entender más contexto del que debería.

Deployments todo-o-nada

Si el equipo de pagos quiere hacer un hotfix urgente, debe desplegar también todo el código de usuarios, eventos y tickets — aunque no haya cambiado nada en esos módulos. Cada deploy es un evento de riesgo completo, independientemente de cuán pequeño sea el cambio.

Fault propagation

Un error no controlado en un módulo puede derribar el proceso completo. Si el módulo de búsqueda de eventos entra en un loop de consumo excesivo de memoria, puede causar un OOM que mata también el módulo de pagos. En un monolito, los módulos comparten destino.

Escalado horizontal limitado

Si el módulo de búsqueda está saturado, la única opción es escalar todo el monolito: levantar más instancias del proceso completo, con todo su consumo de CPU y memoria, aunque el cuello de botella sea solo ese módulo. Esto desperdicia recursos y tiene un límite práctico.

Barreras tecnológicas

Todos los módulos deben usar el mismo lenguaje, la misma versión del runtime y las mismas dependencias. Si un módulo se beneficiaría enormemente de una biblioteca solo disponible en Python, o si el equipo de un

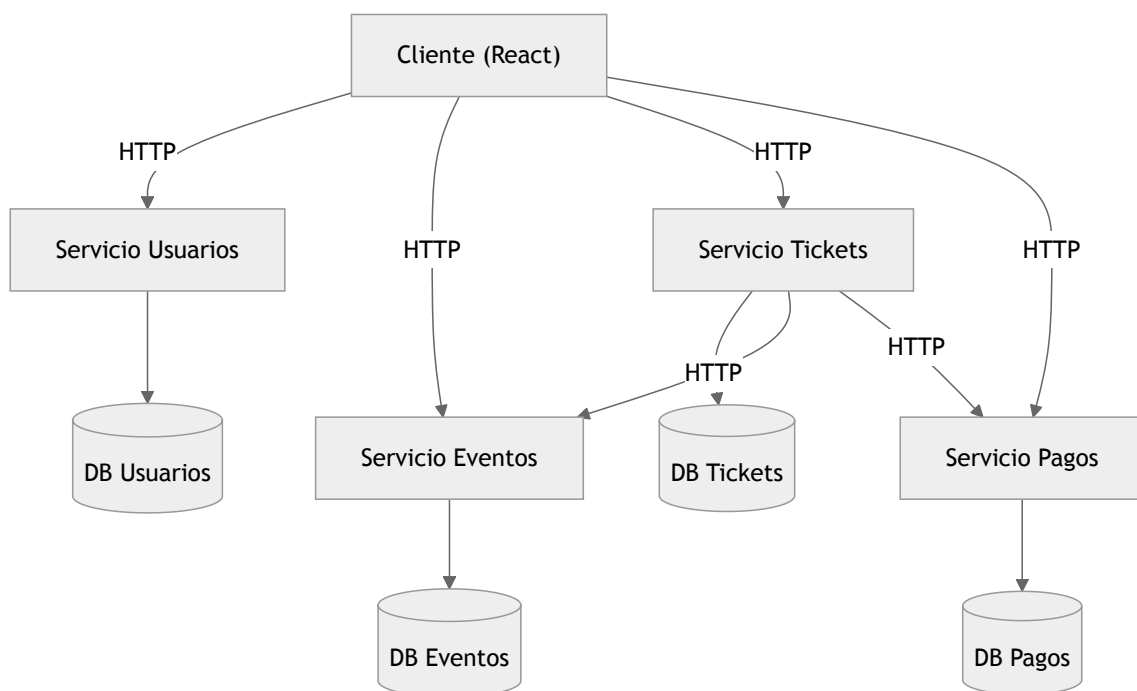
módulo quiere actualizar una dependencia que rompe la interfaz que usa otro módulo, el monolito lo impide.

3. Microservicios

3.1 Definición y principios

Una arquitectura de **microservicios** es un sistema compuesto por múltiples servicios autónomos, delimitados alrededor de capacidades de negocio concretas, que se comunican entre sí por red.

La palabra “micro” es engañosa — no se refiere al tamaño del código sino al alcance de la responsabilidad. Un microservicio hace una cosa y la hace bien. Puede tener miles de líneas de código si el dominio lo requiere.



Los principios que definen una arquitectura de microservicios son:

Límites de dominio: cada servicio encapsula una capacidad de negocio completa, incluyendo sus datos. Nadie accede directamente a la base de datos de otro servicio — si necesitas un dato de otro dominio, lo pedís a través de su API.

Autonomía: el servicio puede desplegarse y escalarse de forma independiente. El aislamiento ante fallos es una capacidad que debe diseñarse mediante límites, timeouts y degradaciones; no aparece automáticamente por separar procesos.

Interfaces explícitas: la única forma de interacción entre servicios es a través de contratos explícitos — APIs REST, eventos, mensajes. No hay llamadas a funciones internas de otro servicio, no hay acceso a su base de datos.

Deploy independiente: cada servicio tiene su propio pipeline de CI/CD. El equipo de tickets puede desplegar una nueva versión sin coordinar con el equipo de eventos.

3.2 Beneficios

Escalado independiente: si el servicio de tickets está saturado en la semana de venta de entradas para un festival, se escalan solo las instancias de ese servicio — no se toca el servicio de usuarios ni el de pagos.

Potencial de aislamiento de fallos: un error en el servicio de búsqueda de eventos no tiene por qué tumbar el servicio de pagos. El aislamiento real, sin embargo, depende de cómo se diseñen las dependencias entre servicios y cómo se manejen las degradaciones — no es automático por el hecho de ser microservicios.

Libertad tecnológica: el servicio de búsqueda puede usar Elasticsearch y Python si el dominio lo justifica. El servicio de pagos puede seguir en Go. Cada equipo elige la herramienta más adecuada para su problema.

Desarrollo en paralelo: equipos distintos pueden trabajar sobre servicios con límites claros y ciclos de entrega independientes. Esto reduce la coordinación y los conflictos, tanto si se usan repositorios separados como un monorepo.

Deployments de menor riesgo: cada deploy afecta solo el servicio que cambió. Un bug en producción tiene un alcance acotado y el rollback es más simple.

3.3 Costos y complejidad

Los microservicios resuelven problemas reales, pero introducen una clase de problemas que no existían en el monolito. Es importante entenderlos antes de decidir migrar.

Overhead operacional: con 8 servicios hay 8 procesos para desplegar, 8 para monitorear, 8 para configurar, 8 para actualizar dependencias de seguridad. La infraestructura que antes era trivial ahora requiere inversión.

Latencia de red: lo que antes era una llamada de función en nanosegundos ahora es una llamada HTTP que puede tardar decenas de milisegundos y puede fallar. Una operación que encadena 5 servicios puede acumular latencia significativa.

Consistencia distribuida: en el monolito, una operación que toca usuarios, tickets y pagos puede ejecutarse en una única transacción ACID. En microservicios esa garantía desaparece: cada servicio gestiona su propia base de datos y no existe un mecanismo nativo que asegure que un conjunto de operaciones distribuidas se complete de forma atómica. Si un servicio falla a mitad de una operación que otro ya ejecutó, el sistema puede quedar en un estado inconsistente. Gestionar esa consistencia requiere estrategias de coordinación explícitas que añaden complejidad al diseño.

***Nota:** Las estrategias concretas para coordinar transacciones entre servicios y gestionar la consistencia eventual — incluyendo el patrón Saga — se estudian en detalle en la **Clase 5** (Datos en microservicios) y la **Clase 7** (Comunicación asíncrona).*

Trazabilidad y debugging distribuido: cuando una operación falla, el error puede originarse en cualquiera de los servicios que participaron. Para rastrear el camino completo de una request a través de múltiples servicios se necesita *distributed tracing*: una técnica que requiere propagar un identificador único de petición — el

Correlation ID — en los headers HTTP o gRPC de cada llamada entre servicios, de forma que las herramientas de observabilidad puedan reconstruir el flujo completo de extremo a extremo.

Más puntos de fallo: cada comunicación entre servicios puede fallar por timeout, error de red o caída del servicio destino. La aplicación debe manejar estos fallos de forma explícita y confiable; en un monolito también hay errores, pero no existe esta clase de fallo de red entre módulos.

3.4 Condiciones mínimas antes de distribuir

Los microservicios no solo agregan código: multiplican el sistema que hay que operar. Antes de extraer servicios, el equipo debería contar como mínimo con:

- **Despliegues automatizados y reversibles:** pipelines de CI/CD que permitan construir, probar, desplegar y volver a una versión conocida.
- **Observabilidad centralizada:** logs, métricas, alertas y trazabilidad distribuida para diagnosticar qué ocurre entre servicios.
- **Gestión de configuración y secretos:** una forma segura y consistente de administrar credenciales, variables de entorno y configuraciones por ambiente.
- **Contratos versionados:** APIs y eventos con compatibilidad hacia atrás mientras los consumidores se actualizan.

No es necesario dominar estas herramientas antes de la primera extracción, pero sí reconocer que son parte del costo y de la madurez operacional que exige una arquitectura distribuida.

4. Trade-offs: cuándo migrar (y cuándo no)

La arquitectura de microservicios no es un objetivo en sí mismo. Es una solución a problemas específicos que emergen cuando un sistema y su equipo crecen. Adoptarla antes de que esos problemas existan es pagar un costo sin recibir el beneficio.

4.1 El monolito no es el enemigo

Hay una tendencia en la industria a asociar monolito con deuda técnica y microservicios con modernidad. Es una simplificación incorrecta. Empresas como Stack Overflow sirven millones de usuarios con una arquitectura monolítica bien mantenida. El monolito es la elección correcta para:

- Equipos pequeños (menos de 8-10 personas en el backend)
- Sistemas en etapa de definición del producto, donde los límites de dominio aún no están claros
- Dominios donde la consistencia transaccional es crítica y compleja de replicar en un sistema distribuido
- Organizaciones sin la madurez operacional para mantener múltiples servicios en producción

4.2 Señales de que llegó el momento de migrar

No hay una regla universal, pero estas señales suelen indicar que los costos del monolito empiezan a superar sus beneficios:

- **El tiempo de deploy bloquea la productividad:** si desplegar un cambio pequeño requiere esperar una build de 20 minutos y un proceso de QA completo de todo el sistema, el equipo ralentiza su cadencia de entrega.
- **Un módulo necesita escalar independientemente:** el tráfico en un módulo es órdenes de magnitud mayor que en los demás, pero se está escalando todo el sistema para satisfacerlo.
- **El equipo creció y los conflictos son frecuentes:** múltiples desarrolladores modificando el mismo repositorio a la vez generan conflictos de merge y dependencias implícitas entre cambios paralelos.
- **Un área necesita una tecnología distinta:** el módulo de machine learning se beneficiaría de Python, pero el monolito es Go y cambiar el lenguaje no es una opción.
- **Los fallos se propagan entre módulos sin relación:** un problema en el módulo de recomendaciones causa lentitud en el proceso de compra, aunque no haya dependencia lógica entre ellos.

4.3 El error de migrar demasiado temprano

La complejidad operacional de los microservicios tiene un costo fijo: hay que pagarla aunque el sistema sea pequeño. Si un equipo de 4 personas divide su monolito en 8 microservicios, ahora tiene que operar 8 servicios — y eso puede consumir más tiempo que el que se ahorra en los deployments.

Martin Fowler lo resume así: *“Don’t start a new project with microservices, even if you’re sure your application will be big enough to make it worthwhile.”* La recomendación estándar es empezar con un monolito bien modularizado y extraer servicios cuando haya una razón concreta para hacerlo.

4.4 El paso intermedio: mejorar la modularidad

Antes de decidir distribuir el sistema, vale la pena preguntarse si el problema actual se puede resolver mejorando la modularidad interna del monolito. Si el acoplamiento entre módulos es el problema, la solución puede ser introducir interfaces claras entre ellos — sin cambiar la arquitectura de despliegue. El monolito modular reduce la fricción de desarrollo sin agregar la complejidad operacional de los servicios distribuidos.

5. Estrategias de migración

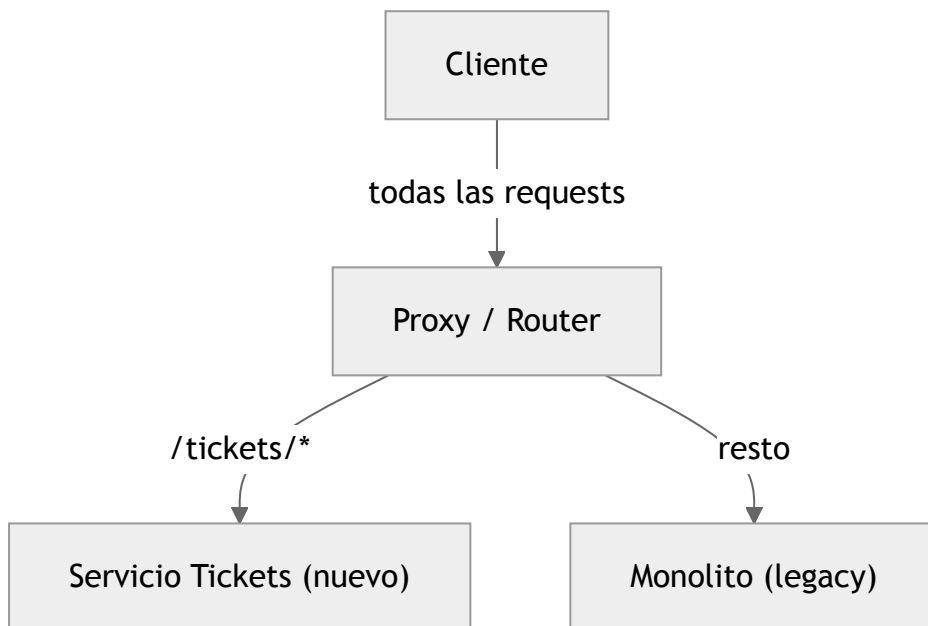
Supongamos que el análisis anterior confirmó que es momento de migrar. El siguiente problema es cómo hacerlo. La respuesta ingenua — reescribir todo de cero y hacer un cutover — tiene un nombre en la industria: *Big Bang Rewrite*, y su historial de fracasos es extenso.

Las estrategias que siguen son alternativas incrementales: permiten migrar el sistema mientras sigue en producción, reduciendo el riesgo a cada paso.

5.1 Strangler Fig Pattern

El nombre viene de un árbol tropical que crece envolviendo a otro árbol huésped. Con el tiempo, el árbol estrangulador reemplaza completamente al huésped sin que ninguno de los dos haya dejado de existir abruptamente.

El patrón aplica la misma lógica a la migración de software: en lugar de reemplazar el monolito de golpe, se extrae una funcionalidad a la vez hacia un nuevo servicio, mientras el monolito sigue sirviendo el resto. Bien aplicado, permite mantener el sistema disponible durante gran parte de la transición.



El componente central es el **proxy o router de borde**: intercepta todas las requests entrantes y decide si las despacha al monolito o al nuevo servicio, según la ruta o el recurso involucrado. Al principio, el proxy envía todo al monolito. A medida que se extraen módulos, el proxy redirige más rutas al nuevo servicio. Cuando el último módulo está migrado, el monolito puede darse de baja.

En implementaciones más maduras, este componente evoluciona hacia un **API Gateway** con responsabilidades adicionales como autenticación centralizada, rate limiting y service discovery. El diseño y las funcionalidades avanzadas del API Gateway se estudian en detalle en la **Clase 3**.

Ventajas: - Permite mantener el servicio disponible mientras se reemplazan funcionalidades de manera gradual - El enrutamiento puede volver al monolito si el nuevo servicio falla; el rollback debe planificarse, especialmente cuando hay datos migrados - El riesgo está acotado al módulo que se está migrando en ese momento - Permite validar el nuevo servicio con tráfico real antes de comprometerse completamente

Desafío clave: los datos

El mayor problema al extraer un módulo es la base de datos. En el monolito, todos los módulos comparten la misma base de datos y pueden hacer JOINS entre tablas. Cuando se extrae el módulo de Tickets a un servicio independiente, ese servicio necesita su propia base de datos — y el monolito ya no puede hacer un JOIN entre la tabla de tickets y la de eventos.

Las consecuencias son concretas: - Las Foreign Keys entre tablas de dominios distintos deben reemplazarse por referencias lógicas. Según el caso, se resuelven mediante una API o mediante una proyección local actualizada por eventos. - Las operaciones que antes eran una sola transacción ahora deben coordinarse entre dos sistemas. - Hay que decidir cómo migrar los datos existentes sin inconsistencias. - El nuevo servicio debe ser dueño de sus datos: otros servicios no deberían leer ni escribir sus tablas directamente.

Este problema de separación de datos — conocido como *Database per Service* — no exige necesariamente un servidor ni un motor de base de datos distinto por servicio. Puede comenzar con esquemas separados y ownership claro. Se desarrolla en profundidad en la Clase 5.

5.2 Feature Flags

Un **feature flag** (también llamado feature toggle) es un mecanismo para habilitar o deshabilitar en producción un comportamiento que ya fue desplegado, sin requerir un nuevo deploy para activarlo o desactivarlo. Se trata de una condición evaluada en tiempo de ejecución:

```
if featureFlags.IsEnabled("nuevo-servicio-tickets", userID) {  
    // llamar al nuevo servicio de Tickets  
} else {  
    // usar la implementación del monolito  
}
```

En el contexto de una migración, los feature flags permiten:

- **Rollout gradual:** habilitar el nuevo comportamiento para un porcentaje del tráfico e ir aumentando ese porcentaje a medida que se gana confianza.
- **Rollback instantáneo:** si algo falla en el nuevo servicio, deshabilitar el flag en segundos — sin necesidad de un deploy de emergencia.
- **Testing en producción con usuarios reales:** habilitar el nuevo comportamiento solo para usuarios internos o para un grupo de beta testers antes de abrirlo al público general.

Tipos de flags:

Tipo	Descripción	Ejemplo de uso
Por entorno	Activo en staging, inactivo en producción	Probar un cambio en staging antes de habilitarlo
Por usuario	Activo solo para ciertos user IDs	Dar acceso anticipado a usuarios internos
Por porcentaje	Activo para el X% del tráfico	Rollout gradual del 1% al 100%
Por región	Activo solo en determinados países	Validar en un mercado pequeño antes de expandir

Relación con Strangler Fig: el proxy del patrón Strangler Fig puede usar feature flags para controlar qué porcentaje del tráfico se envía al nuevo servicio. El proxy evalúa el flag en cada request y decide el destino. Esto permite que la migración sea gradual y reversible, no un corte abrupto.

Deuda técnica de los flags: los feature flags que no se limpian después de la migración se convierten en deuda técnica. El código con un flag que siempre está activo es código muerto disfrazado. Una disciplina de migración con flags incluye una fecha de expiración para cada flag y la tarea de eliminar el código de la rama desactivada una vez que la migración está validada.

5.3 Branch by Abstraction

Este patrón resuelve un problema diferente al Strangler Fig: cómo cambiar una implementación interna del monolito sin romper a todos los que la usan, especialmente cuando el cambio es gradual.

El proceso tiene cuatro pasos:

1. Introducir la abstracción: definir una interfaz que represente el comportamiento del componente que se quiere reemplazar. El código consumidor pasa a depender de esa interfaz, no de la implementación concreta.

```
// Antes: dependencia directa
type TicketService struct {
    repo *TicketRepository // implementación concreta
}

// Después: dependencia de la abstracción
type TicketStorage interface {
    Save(ticket Ticket) error
    FindByEvent(eventID string) ([]Ticket, error)
}

type TicketService struct {
    storage TicketStorage // abstracción
}
```

2. Incorporar la nueva implementación: agregar la nueva implementación detrás de la interfaz. El código existente no cambia — sigue usando la abstracción.

3. Redirigir gradualmente: usando un feature flag u otro mecanismo, ir cambiando instancias del código para que usen la nueva implementación en lugar de la vieja.

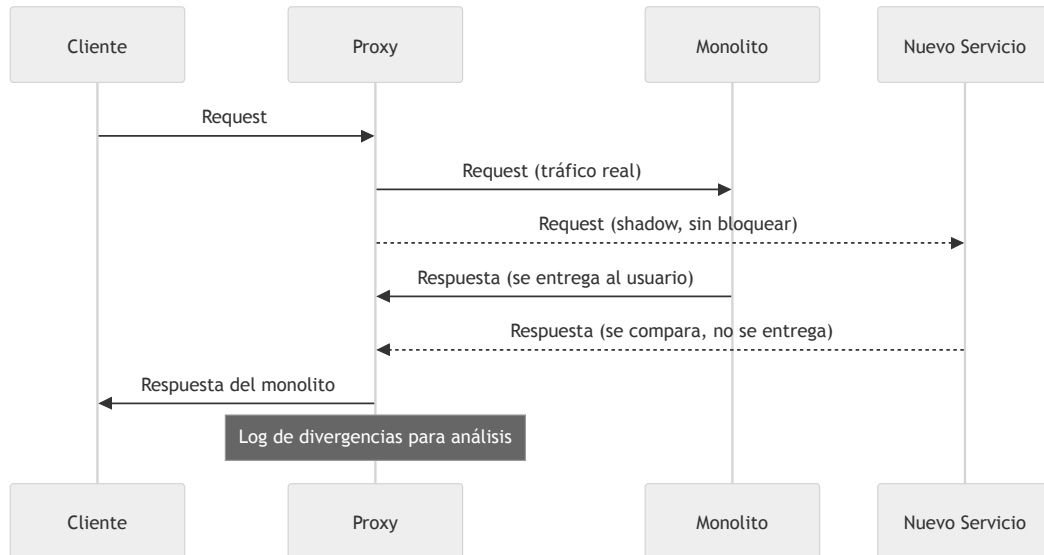
4. Eliminar la implementación anterior: cuando la nueva implementación está completamente adoptada y validada, se elimina la vieja y, si ya no hace falta, también la abstracción.

Cuándo usarlo: es especialmente útil cuando se quiere extraer un componente del monolito y convertirlo en un servicio independiente, pero el componente tiene muchos consumidores dentro del mismo codebase. La abstracción permite hacer la transición sin tocar todos los puntos de uso de una vez.

Relación con Feature Flags: el flag puede decidir qué implementación usar mientras ambas coexisten, controlando la transición de forma gradual.

5.4 Parallel Run / Shadow Traffic

En esta estrategia, el sistema nuevo procesa las mismas requests que el sistema existente, pero su respuesta **no se entrega al usuario**. En cambio, se compara con la respuesta del sistema existente para detectar divergencias.



Objetivo: ganar confianza en que el nuevo servicio produce los mismos resultados que el sistema existente, antes de exponerlo a tráfico real. Es una forma de testing en producción sin riesgo para el usuario.

Consideraciones: - **Efectos secundarios:** si el nuevo servicio escribe en una base de datos o envía notificaciones, el shadow traffic puede generar efectos no deseados (duplicación de datos, emails enviados dos veces). Hay que asegurarse de que el nuevo servicio sea seguro de invocar en modo shadow. -

Comparación de respuestas: las respuestas pueden diferir en orden de elementos, timestamps, o campos irrelevantes. El comparador debe ser lo suficientemente inteligente para ignorar diferencias que no importan. -

Costo de infraestructura: el sistema está procesando cada request dos veces. Esto tiene un costo de cómputo que hay que evaluar.

Caso en EventOS: antes de derivar tráfico real al nuevo Servicio de Tickets, se envían en modo shadow las consultas de disponibilidad y los cálculos de precio. Si los resultados coinciden durante un período de tiempo, se gana la confianza necesaria para iniciar un rollout gradual de las compras reales.

5.5 Expand and Contract

Este patrón resuelve el problema de cambiar una API o un esquema de base de datos cuando hay múltiples consumidores que no pueden actualizarse todos al mismo tiempo. Es especialmente relevante durante la migración, cuando el monolito y el nuevo servicio coexisten y pueden tener expectativas distintas sobre los datos.

El principio subyacente es la **compatibilidad hacia atrás** (*backward compatibility*): mientras existan consumidores que dependen de una estructura — un campo en una respuesta JSON, una columna en una tabla, un endpoint — esa estructura no puede eliminarse ni modificarse de forma incompatible. Cambiarla sin

respetar esa dependencia se denomina *breaking change* y puede romper a todos los consumidores que no fueron actualizados.

Cambios no rompedores (seguros durante la fase Expand): - Agregar un nuevo endpoint sin eliminar el existente - Agregar campos opcionales a una respuesta JSON - Agregar una nueva columna nullable a una tabla

Cambios rompedores (solo permitidos en la fase Contract): - Eliminar un campo o columna existente - Renombrar un campo, columna o endpoint - Modificar el tipo de dato de un campo existente

El patrón tiene tres fases:

Expand (expandir): agregar las nuevas estructuras — campos, tablas, endpoints — sin eliminar las existentes. Ambas versiones coexisten. Los consumidores viejos siguen funcionando con la estructura anterior; los nuevos adoptan la nueva.

```
-- Expand: agregar columna nueva sin eliminar la vieja
ALTER TABLE tickets ADD COLUMN external_id VARCHAR(36);
-- La columna vieja sigue existiendo para los consumidores que la usan
```

Migrate (migrar): actualizar los consumidores para que usen la nueva estructura. Esto puede hacerse gradualmente, servicio por servicio o módulo por módulo.

Contract (contraer): cuando todos los consumidores adoptaron la nueva estructura y ninguno usa ya la vieja, se aplican los cambios rompedores y se elimina la estructura anterior.

```
-- Contract: eliminar la columna vieja, ya sin consumidores activos
ALTER TABLE tickets DROP COLUMN old_id;
```

Relación con la extracción de servicios: cuando se extrae un módulo del monolito, el nuevo servicio puede necesitar un esquema diferente al que tiene el monolito. El patrón Expand and Contract permite hacer esa transición de forma que ambos sistemas operen simultáneamente durante la migración, sin romper ninguno de los dos.

Durante la coexistencia debe mantenerse un único dueño de cada dato. Si temporalmente es necesario copiar o sincronizar información, hay que definir cuál es la fuente de verdad y cómo se detectarán o resolverán divergencias.

5.6 Big Bang Rewrite

El *Big Bang Rewrite* es reemplazar el sistema — o una parte significativa de él — en un único corte. Se construye el nuevo sistema en paralelo, y en una fecha determinada se apaga el viejo y se enciende el nuevo.

Este enfoque **concentra toda la incertidumbre en el momento del cambio**: incertidumbre técnica (¿funciona el nuevo sistema?), funcional (¿replica todo el comportamiento del viejo?), y operativa (¿puede manejar el tráfico de producción?). Si algo falla, el rollback es costoso o imposible.

La industria tiene ejemplos notorios de Big Bang Rewrites que fallaron. Netscape reescribió su browser desde cero y perdió el mercado frente a Internet Explorer. El proyecto Denali de eBay fue cancelado después de años de trabajo. En ambos casos, el tiempo que tomó la reescritura permitió que los competidores avanzaran.

¿Cuándo podría justificarse?

- El sistema es pequeño y el riesgo de la migración incremental es mayor que el de un corte limpio.
- Hay una ventana de tiempo clara con bajo tráfico (un fin de semana, una temporada baja) que permite hacer el cambio y validar sin presión.
- La arquitectura del sistema viejo es tan caótica que no hay forma de aplicar los patrones incrementales sin una inversión que supera el costo del rewrite.

Aun en estos casos, es la alternativa de mayor riesgo y no la opción por defecto. La recomendación estándar es agotar las estrategias incrementales antes de considerar el Big Bang.

5.7 Cuadro comparativo de estrategias

Estrategia	Ámbito de aplicación	Mecanismo central	Nivel de riesgo
Strangler Fig	Extracción de módulos del monolito hacia servicios	Proxy/router que redirige tráfico gradualmente	Variable — el enrutamiento es reversible; los datos requieren un plan propio
Feature Flags	Control del rollout de cualquier cambio	Flag evaluado en runtime que activa/desactiva comportamiento	Bajo para cambiar el comportamiento; no revierte efectos ya realizados
Branch by Abstraction	Cambio de implementación interna con múltiples consumidores	Interfaz que desacopla consumidores de la implementación	Bajo — el cambio es interno al sistema
Parallel Run / Shadow Traffic	Validación del nuevo sistema antes del cutover	Procesamiento duplicado; la respuesta del nuevo sistema no se entrega al usuario	Bajo — el usuario no es impactado
Expand and Contract	Cambios de API o esquema con múltiples consumidores	Coexistencia temporal de la estructura vieja y la nueva	Bajo — ambas versiones funcionan en simultáneo
Big Bang Rewrite	Reemplazo completo del sistema	Corte único entre el sistema viejo y el nuevo	Alto — toda la incertidumbre se concentra en el cambio

Estas estrategias no son mutuamente excluyentes — en una migración real suelen combinarse. Una migración típica puede usar Strangler Fig como estrategia general, Feature Flags para controlar el rollout del tráfico,

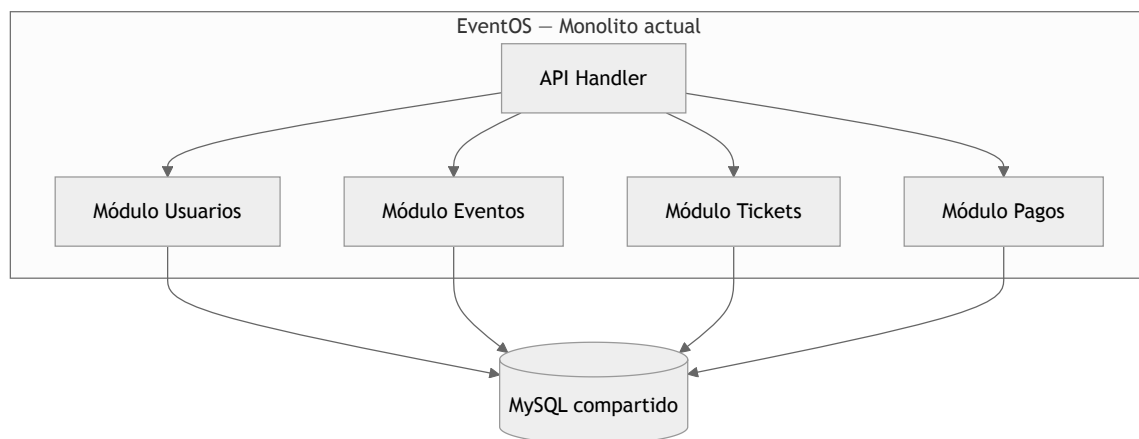
Expand and Contract para manejar los cambios de esquema, y Parallel Run para validar el nuevo servicio antes del cutover definitivo.

Una posible secuencia de migración es:

1. Delimitar la capacidad a extraer y definir el ownership de sus datos.
2. Aplicar **Expand and Contract** para preparar estructuras compatibles y migrar la información.
3. Usar **Branch by Abstraction** si muchos consumidores internos dependen de la implementación actual.
4. Validar lecturas o cálculos sin efectos secundarios mediante **Parallel Run / Shadow Traffic**.
5. Habilitar escrituras reales gradualmente con un **Feature Flag** y un rollout canary.
6. Derivar la ruta al nuevo servicio mediante **Strangler Fig** y retirar la implementación o los datos anteriores cuando ya no tengan consumidores.

6. EventOS: aplicación práctica

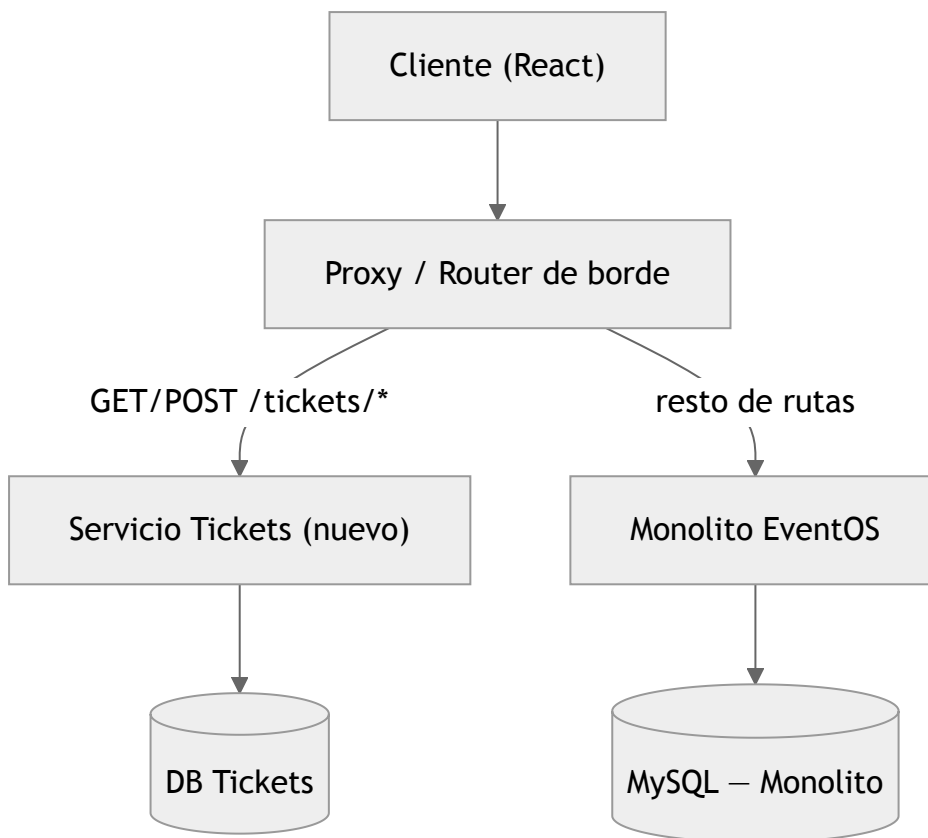
EventOS es una plataforma de venta de tickets online. Permite a los organizadores publicar eventos y a los usuarios comprar entradas. Tiene cuatro dominios principales: Usuarios, Eventos, Tickets y Pagos. En su versión inicial fue construida como un monolito: un único proceso Go con una base de datos MySQL compartida.



El problema a escala: EventOS creció. Hay 200.000 usuarios registrados. Cuando hay un festival importante, el módulo de Tickets recibe un pico de tráfico que es 50 veces mayor que el tráfico habitual. La única opción hoy es escalar todo el monolito — levantar más instancias de todo el proceso — aunque el cuello de botella sea solo Tickets. El costo de infraestructura es enorme. Además, el equipo de Pagos quiere integrar un nuevo proveedor de pagos, pero cada vez que deployean ese cambio tienen que coordinar con el equipo de Eventos, que también quiere desplegar mejoras en la búsqueda. Los deployments se bloquean mutuamente.

El plan de migración con Strangler Fig:

El primer módulo a extraer es Tickets, porque es el que tiene el pico de tráfico más pronunciado y el beneficio del escalado independiente es inmediato.



El proxy de borde redirige todas las rutas `/tickets/*` al nuevo servicio. El resto sigue yendo al monolito. El nuevo Servicio de Tickets tiene su propia base de datos — lo que implica resolver el problema de la consistencia con los datos de Eventos que siguen en el monolito.

Durante la migración, se usa Parallel Run para operaciones seguras, como consultar disponibilidad o calcular el precio de una entrada. Esas requests se procesan tanto en el monolito como en el nuevo servicio y se comparan sus respuestas. Las compras reales no se duplican en modo shadow: pueden reservar stock, cobrar o generar notificaciones. Una vez validado el servicio, las escrituras se habilitan gradualmente con un feature flag, enviando cada compra a un único camino de procesamiento.

7. Trade-offs y decisiones arquitectónicas

La decisión de migrar a microservicios no tiene una respuesta correcta universal. Depende del contexto: tamaño del equipo, madurez operacional, naturaleza del tráfico, criticidad del dominio. Lo que sí es posible definir son las tensiones que hay que resolver:

Tensión	Monolito	Microservicios
Consistencia de datos	Transacciones locales simples cuando comparten recurso	Coordinación entre datos distribuidos; a menudo consistencia eventual
Tiempo de desarrollo inicial	Más rápido	Más lento (infraestructura, contratos de API)
Escalado	Todo el sistema	Por servicio
Debugging	Directo, local	Requiere distributed tracing
Deploy	Un artefacto, más riesgo por cambio	Múltiples artefactos, menor riesgo por cambio
Complejidad operacional	Baja	Alta
Autonomía de equipos	Baja (conflictos de merge, coordinación)	Alta (equipos independientes por servicio)
Latencia interna	Llamada de función (~ns)	Llamada de red (~ms)

No hay un ganador absoluto. La arquitectura correcta es la que resuelve el problema real del sistema en su etapa actual, con el equipo y los recursos disponibles.

8. Puntos clave

- Un **monolito** es una única unidad de despliegue. No implica mala calidad ni falta de organización — un monolito bien modularizado es una arquitectura válida y frecuentemente la mejor opción.
- Los **límites del monolito a escala** son concretos: deployments todo-o-nada, fault propagation, escalado indiferenciado y acoplamiento que crece con el tiempo.
- Los **microservicios** son servicios autónomos organizados alrededor de capacidades de negocio, con interfaces explícitas y ciclos de vida independientes.
- Los beneficios de los microservicios —escalado independiente, aislamiento de fallos, autonomía de equipos— solo se materializan con un diseño cuidadoso. No son automáticos.
- Los costos de los microservicios son reales: complejidad operacional, latencia de red, consistencia distribuida y debugging más complejo.
- **Migrar demasiado temprano** es un error común: la complejidad operacional de los microservicios tiene un costo fijo que puede superar los beneficios en sistemas pequeños.
- El **Strangler Fig Pattern** es la estrategia de migración incremental estándar: extraer módulos uno a la vez usando un proxy, mientras el monolito sigue en producción.
- Los **Feature Flags** permiten controlar el rollout de cada cambio con granularidad y desactivar rápidamente un comportamiento, aunque no revierten efectos ya realizados.

- **Branch by Abstraction** desacopla a los consumidores de una implementación concreta, permitiendo reemplazarla gradualmente.
 - **Parallel Run / Shadow Traffic** valida el nuevo sistema con tráfico real antes de exponerlo al usuario.
 - **Expand and Contract** permite cambiar APIs y esquemas de base de datos de forma retrocompatible durante la coexistencia.
 - El **Big Bang Rewrite** concentra toda la incertidumbre en un único evento y es la alternativa de mayor riesgo. Raramente es la mejor opción.
 - Las estrategias de migración se **combinan**: una migración real típicamente usa varias en simultáneo.
-

9. Referencias y profundización

- [Microservices — Martin Fowler](#) — El artículo que popularizó el término. Lectura obligatoria para entender el contexto histórico y la definición original.
- [Strangler Fig Application — Martin Fowler](#) — Descripción original del patrón por quien lo nombró.
- [Feature Toggles — Pete Hodgson](#) — Guía completa sobre tipos de flags, ciclo de vida y gestión de la deuda técnica que generan.
- [Branch By Abstraction — Paul Hammant](#) — Descripción del patrón con ejemplos concretos.
- [Monolith First — Martin Fowler](#) — Por qué empezar con un monolito es casi siempre la mejor decisión para sistemas nuevos.
- [Building Microservices — Sam Newman](#) — El libro de referencia de la industria sobre arquitectura de microservicios.
- [Database per Service — Chris Richardson](#) — Ownership de datos, esquemas separados y desafíos de consultas y transacciones entre servicios.
- [Saga — Chris Richardson](#) — Patrón para coordinar transacciones distribuidas mediante transacciones locales y acciones compensatorias.
- [Strangler Fig Pattern — Microsoft Azure Architecture Center](#) — Explicación actualizada de la fachada de enrutamiento, la migración progresiva y el retiro del sistema legado.
- [Microservice Architecture Pattern Language — Chris Richardson](#) — Mapa de patrones de descomposición, datos, comunicación, observabilidad y despliegue para profundizar durante la materia.